POLITECNICO
MILANO 1863

Optimal Control for spacecraft Nadir pointing

Numerical Optimization for Control

Academic Year 2024-2025

Berti Lorenzo 10533775
Giaquinto Elisabetta 10605230
Mikhalev Aleksandr 10960273
Vurro Martina Angela 11009775

Contents

1 Introduction **1**

2 Spacecraft Attitude and Dynamics **2**
2.1 Model Assumptions . . . 2
2.2 Spacecraft and Orbit selection . . . 2
2.3 Reference Frames . . . 3
2.4 Dynamics . . . 4
2.5 Disturbing Torques . . . 4
2.6 Control Torque . . . 5
2.7 Kinematics . . . 5
2.8 Nadir Pointing . . . 6

3 Problem Abstraction **7**
3.1 FHOCP with torque input . . . 7
3.2 FHOCP with PID controller . . . 9
3.2.1 PID design and implementation . . . 10
3.3 Final considerations . . . 11

4 Optimization Procedure **11**
4.1 Solver Selection for Optimization . . . 11
4.2 Algorithms overview . . . 11
4.3 Evaluation of Optimization Algorithms for Given Nonlinear Constrained Problem . 12
4.3.1 Iterations Required for Convergence . . . 13
4.3.2 Function Evaluations . . . 13
4.3.3 Computational Time . . . 14
4.3.4 Final Cost (Objective Function Value) . . . 14
4.3.5 Conclusion . . . 14
4.4 Finite Difference Type Comparison: Forward vs. Central Differentiation . . . 14
4.4.1 Introduction . . . 14
4.4.2 Iterations Required for Convergence . . . 15
4.4.3 Function Evaluations . . . 15
4.4.4 Computational Time . . . 15
4.4.5 Final Cost (Objective Function Value) . . . 15
4.4.6 Conclusion . . . 16
4.5 Other solver settings . . . 16
4.6 Handling Non-Convexity in Optimization . . . 16

5 Results **16**

References **20**

List of Figures

1 MEO Orbit . . . 3
2 Representation of orbital Parameters . . . 3
3 Reference frames: ECI, LVLH, body . . . 4
4 Solar Radiation Pressure . . . 5
5 Gravity Gradient . . . 5
6 Pointing Error . . . 8
7 Quaternion Error . . . 8
8 PID implemented in simulink . . . 10
9 Comparison of Different Algorithms . . . 13

10 Comparison of Finite Difference Type 15
11 Quaternion error without control 17
12 Quaternion error with PID 17
13 Pointing error without control 18
14 Pointing Error with PID 18
15 Angular velocity without control 18
16 Angular velocity with PID 18
17 Control Torque 19

List of Tables

1 12U features 2
2 Orbital Parameters 2
3 Main drawbacks of FHOCP with input torque and $t_s = 20$ s 8
4 Common algorithms settings 12
5 Final Optimization Results from fmincon 17

1 Introduction

Many space missions, such as Earth observation, remote sensing, and communication satellite operations, require precise attitude control to maintain a fixed orientation relative to the Earth's surface. Nadir pointing, where a spacecraft's specific axis (usually the z-axis) continuously points towards the Earth's center, is essential for these missions, ensuring that instruments or antennas remain directed at the Earth's surface throughout the satellite's orbit. The aim of this report is to simulate the attitude dynamics of a spacecraft in a Medium Earth Orbit (MEO) while ensuring Nadir pointing throughout its operational lifetime. The simulation begins with the spacecraft in a realistic random initial orientation. A slew maneuver is then executed to achieve proper alignment, followed by a pointing routine to maintain the desired orientation. A PID controller is implemented, with its parameters optimized through an iterative optimization process. This ensures not only zero pointing error but also compliance with actuator saturation limits and angular velocity constraints. The simulation is carried out using a Simulink model, with the optimization process implemented in a MATLAB script that updates the model parameters at each iteration.

2 Spacecraft Attitude and Dynamics

2.1 Model Assumptions

The mathematical model developed for the optimization problem accounts for several assumptions:

- A restricted two-body problem is considered, where only the spacecraft and Earth are included, while the influence of other celestial bodies, such as the Sun or other planets, is neglected. The spacecraft's mass is assumed to be negligible compared to Earth, meaning it does not affect Earth's motion.
- The optimization is performed to complete a slew maneuver and maintaining the required pointing. The detumbling phase is not considered, meaning the spacecraft starts with a small initial angular velocity.
- Typical slew maneuvers involve rotations of up to 90° in roll, pitch, and yaw (Ref. [8]). The initial spacecraft attitude is randomly chosen while ensuring these bounds are respected. Roll, which is the rotation around the spacecraft's longitudinal axis, is generally less relevant for pointing objectives. Therefore, a 15° limit is imposed to reflect a realistic scenario, as roll is more commonly used for tasks such as solar panel orientation or momentum management.
- Sensors and actuators are assumed to be perfect and are therefore not explicitly modeled. As a result, the control torque computed by the controller is directly applied to the spacecraft without any delays, noise, or inaccuracies.
- The dynamics is assumed to provide the true spacecraft angular velocity, meaning that an external observer is not required to estimate it.

2.2 Spacecraft and Orbit selection

The spacecraft employed is a 12U CubeSat, assumed to be a rigid body with inertia axes coincident with the symmetry axes. Its geometrical properties are reported in Table 1.

	Mass [kg]	Size [m]
Main body	14.5091 kg	$0.2\times0.2\times0.3$
Solar Panels	0.3628	$0.01\times0.2\times0.3$

Table 1: 12U features

From the geometrical properties, the principal inertia matrix I is computed:

$$I = \begin{bmatrix} 0.2453 & 0 & 0 \\ 0 & 0.1872 & 0 \\ 0 & 0 & 0.1644 \end{bmatrix} [kgm^2] \tag{1}$$

The chosen orbit is an almost circular near-polar Medium Earth Orbit (MEO), defined by the orbital parameters listed in Table 2, which include the semi-major axis (a), eccentricity (e), inclination (i), Right Ascension of the Ascending Node (Ω), argument of periapsis (ω), and initial True Anomaly (θ_0). A graphical representation of these orbital elements is provided in Figure 2.

$a[km]$	$e[-]$	$\Omega[deg]$	$\omega[deg]$	$\theta_0[deg]$
90000	0.01	0	1.5708	0

Table 2: Orbital Parameters

Given the orbital parameters, it is possible to retrieve the orbital period (T_{rev}) and the orbital angular velocity (n), which is the rate at which a spacecraft moves along its orbit around the Earth. Those quantities are computed as follows:

$$n=\sqrt{\left(\frac{\mu}{a^3}\right)}=2.3383\cdot 10^{-5} rad/s \qquad T_{rev}=\frac{2\pi}{n}=268704s \tag{2}$$

In order to propagate the full orbit, a simple restricted two-body problem including the J_2 effect due to oblateness of the Earth is integrated as follows:

$$\frac{d^2\mathbf{r}}{dt^2}=-\mu\frac{\mathbf{r}}{r^3}+\frac{3J_2\mu R_e^2}{2r^4}\left[\frac{x}{r}\left(5\frac{z^2}{r^2}-1\right)\hat{\mathbf{i}}+\frac{y}{r}\left(5\frac{z^2}{r^2}-1\right)\hat{\mathbf{j}}+\frac{z}{r}\left(5\frac{z^2}{r^2}-3\right)\hat{\mathbf{k}}\right] \tag{3}$$

where $\mu=398600km^3/s^2$ is the Earth gravitational parameter, $\mathbf{r}=x\hat{\mathbf{i}}+y\hat{\mathbf{j}}+z\hat{\mathbf{k}}$ is the position vector pointing from the Earth's center to the spacecraft along the orbit, $J_2=0.00108263$ is a constant coefficient and R_e is the Earth's equatorial radius. The initial Cartesian coordinates to perform the propagation are derived from the orbital elements using the standard Keplerian-to-Cartesian transformation, a well-established approach in orbital mechanics (Ref. [4]).

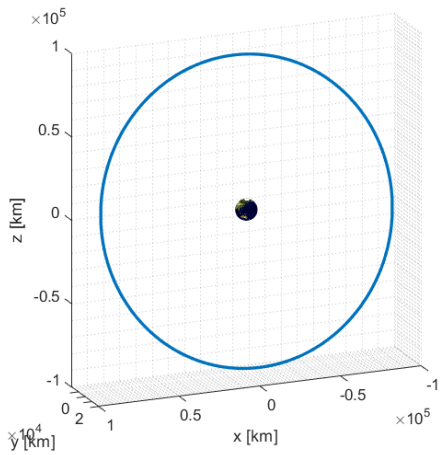

Figure 1: MEO Orbit

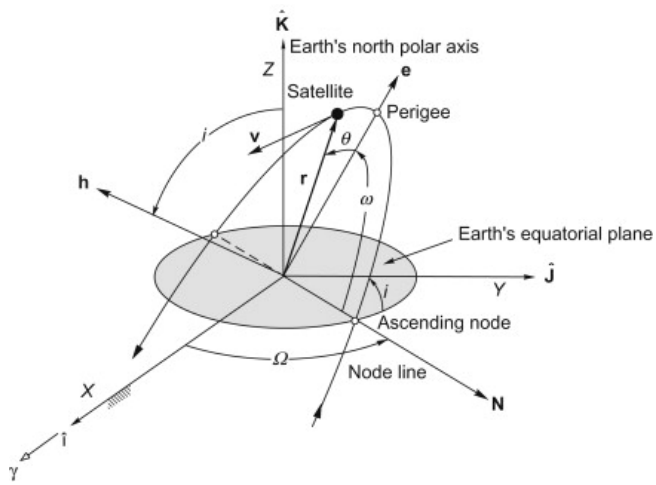

Figure 2: Representation of orbital Parameters

2.3 Reference Frames

Nadir pointing refers to the process of orienting a spacecraft so that a specific axis (typically the z-axis) continuously points toward the Earth's center. To ensure this precise alignment, three different reference frames are used, accounting for the spacecraft's motion and maintaining the desired orientation throughout the mission.

- *Body frame*: it is fixed to the spacecraft and is used to define the spacecraft's orientation in space. Its origin is at the spacecraft's center of mass, and its axes align with the principal axes of the spacecraft.
- *Earth Centered Inertial frame (ECI)*: it is primarily used to describe the spacecraft's orbital dynamics. It is an inertial coordinate system centered at the Earth's center, and it is useful for computing the spacecraft's trajectory and its motion relative to the Earth.
- *Local-Vertical-Local-Horizon (LVLH)*: is a rotating coordinate system where Z-axis points towards the local vertical, i.e., the center of the Earth (Nadir direction); X-axis points along the local horizontal plane, tangential to the Earth's surface; Y-axis completes the right-handed system, perpendicular to both the x and z axes. Since the LVLH frame's z-axis is aligned with the Earth's center, it directly corresponds to the Nadir direction. This makes it the most convenient frame for implementing Nadir pointing since the spacecraft's attitude control system simply needs to maintain the spacecraft's z-axis aligned with this reference.

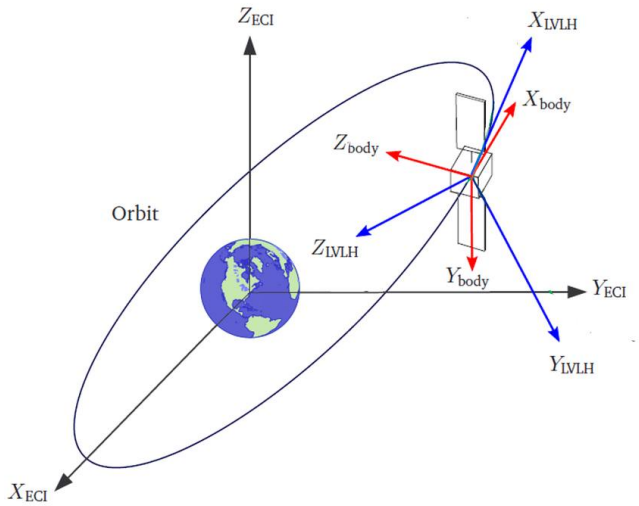

Figure 3: Reference frames: ECI, LVLH, body

2.4 Dynamics

The spacecraft's dynamics in the body reference frame can be computed through the Euler equations, assuming the spacecraft is a rigid body:

$$I\dot{\boldsymbol{\omega}} = I\boldsymbol{\omega} \times \boldsymbol{\omega} + \mathbf{M}_{act} + \mathbf{M}_{dist} \tag{4}$$

where $\boldsymbol{\omega}$ is the spacecraft angular velocity in body frame, $\mathbf{I}$ is the inertia matrix, $\mathbf{M}_{act}$ is the control Torque provided by actuators and $\mathbf{M}_{dist}$ are the disturbing torques.

2.5 Disturbing Torques

Since the orbit is in the MEO regime, the effects of Earth's magnetic field and atmospheric drag are negligible. Therefore, the primary disturbances considered in this model are Solar Radiation Pressure (SRP) and Gravity Gradient (GG), both of which induce disturbing torques that affect the spacecraft's dynamics.

$$\mathbf{M}_{dist} = \mathbf{M}_{GG} + \mathbf{M}_{SRP} \tag{5}$$

The Gravity Gradient torque is due to the fact that the gravity field is not uniform. In the principal inertia reference frame (i.e. body frame) the torque is modeled as:

$$\mathbf{M}_{GG} = \frac{3Gm}{r^3} \begin{bmatrix} (I_z - I_y)c_2c_3 \\ (I_x - I_z)c_1c_3 \\ (I_y - I_x)c_1c_2 \end{bmatrix} \tag{6}$$

where r is the distance between the center of Earth and satellite, m is the mass of Earth, G is the gravitational constant and $[c1,c2,c3]$ are the direction cosines of the radial direction in the principal axes.

The SRP torque is due to the pressure induced on the sunlit surfaces of the satellite due to solar radiation. To compute the torque, the spacecraft is decomposed as a set of flat surfaces, each with a proper diffusion (ρ_d) and specular reflection (ρ_s) coefficients, area and normal versor $\hat{N}_{bi}$. Specifically, for the main body $\rho_s = 0.5, \rho_d = 0.1$ and for solar panels $\rho_s = 0.1, \rho_d = 0.1$. Knowing the Sun's direction in body frame s_{sun_b} and the Earth's direction in body frame s_{earth_b} the torque is evaluated as follows:

$$\mathbf{M}_{SRP} = \begin{cases} \sum_{i=1}^{n_{surface}} \mathbf{r}_i \times \mathbf{F}_i & \hat{S}_b \cdot \hat{N}_{bi} > 0 \\ 0 & \hat{S}_b \cdot \hat{N}_{bi} < 0 \end{cases} \tag{7}$$

where the condition $\hat{S}_b \cdot \hat{N}_{bi} > 0$ implies that the i-th surface is hit by the Sun.

$$\mathbf{F}_i = -PA_i(\hat{S}_b \cdot \hat{N}_{bi})[(1-\rho_s)\hat{S}_b + (2\rho_s(\hat{S}_b \cdot \hat{N}_{bi}) + (2/3)\rho_d)\hat{N}_{bi}] \tag{8}$$

Specifically, $P = F_{sun}/c$ is the pressure induced on the surfaces, c is the speed of light, and $F_{sun} = 1358 W/m^2$ is the direct radiation of the sun.
The evolution of $\mathbf{M_{SRP}}$ and $\mathbf{M_{GG}}$ in an uncontrolled scenario over the full orbital period are shown in Figure 5 and Figure 4.

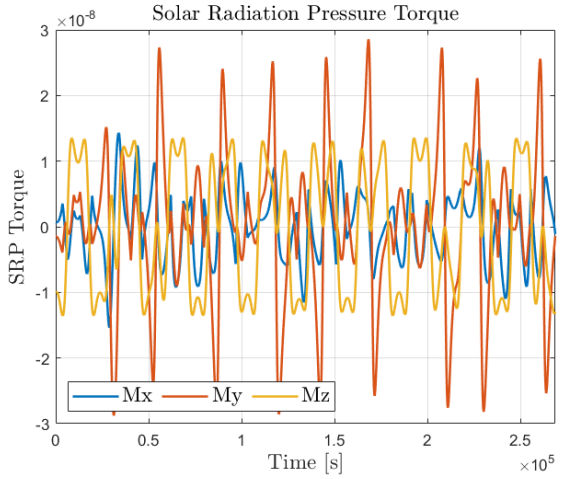

Figure 4: Solar Radiation Pressure

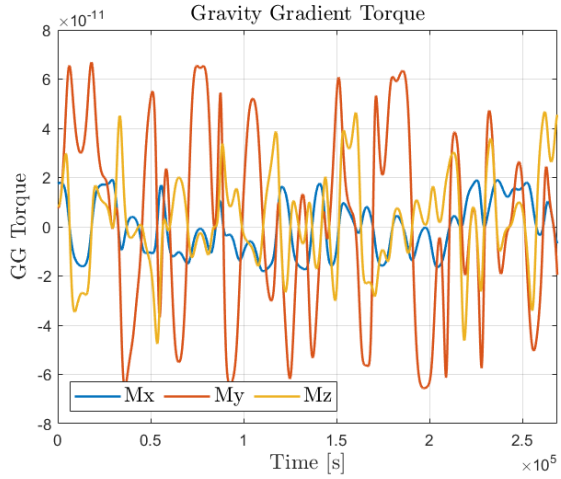

Figure 5: Gravity Gradient

2.6 Control Torque

For agile small satellite missions requiring accurate pointing, three reaction wheels serve as actuators, each capable of providing a maximum torque of 12 mNm, as detailed in the RW400 datasheet [1]. In the model, saturation is conservatively set to $\pm 10 mNm$. For modeling simplicity, the reaction wheels are assumed to be ideal, meaning the control torque generated by the PID controller is applied directly to the spacecraft without errors or deviations typically introduced by actuators, which are not explicitly modeled.

2.7 Kinematics

From the initial conditions and angular velocity obtained from dynamics (Equation 4), the spacecraft's attitude can be determined over time, defining its orientation relative to another frame (e.g., ECI, LVLH). This report employs quaternions to represent attitude. Quaternions are normalized 4-dimensional vectors, consisting in a scalar and a vectorial part (in this report, the last-scalar representation is used, thus the scalar component is the last one).

$$\mathbf{q} = \begin{bmatrix} q_1 & q_2 & q_3 & q_4 \end{bmatrix}^T = \begin{bmatrix} \mathbf{q_v} & q_4 \end{bmatrix}^T \tag{9}$$

Quaternions represent a singularity-free alternative to Euler angles, which, despite their physical interpretability, suffer from gimbal lock, a condition where two rotation axes become aligned, causing a loss of one degree of freedom. Quaternion-based models also ensure numerical stability, computational efficiency, and smooth, continuous rotations, making them ideal for spacecraft attitude control. To describe the evolution of the spacecraft's attitude using quaternions, the kinematic equation is formulated as follows:

$$\dot{\mathbf{q}} = \frac{1}{2}\Omega \mathbf{q} \qquad \text{with} \quad \Omega = \begin{bmatrix} 0 & \omega_z & -\omega_y & \omega_x \\ -\omega_z & 0 & \omega_x & \omega_y \\ \omega_y & -\omega_x & 0 & \omega_z \\ -\omega_x & -\omega_y & -\omega_z & 0 \end{bmatrix} \tag{10}$$

where Ω is the skew-symmetric matrix constructed from the angular velocity vector.
Quaternions can handle successive rotations by using Hamilton product operation as follows:

$$q'' \otimes q' = \begin{bmatrix} q'_4 & -q'_3 & q'_2 & q'_1 \\ q'_3 & q'_4 & -q'_1 & q'_2 \\ -q'_2 & q'_1 & q'_4 & q'_3 \\ -q'_1 & -q'_2 & -q'_3 & q'_4 \end{bmatrix} \begin{bmatrix} q''_1 \\ q''_2 \\ q''_3 \\ q''_4 \end{bmatrix} \tag{11}$$

Moreover, the inverse of a quaternion is defined as:

$$\mathbf{q}^{-1} = \begin{bmatrix} -q_1 & -q_2 & -q_3 & q_4 \end{bmatrix}^T \tag{12}$$

2.8 Nadir Pointing

To achieve Nadir pointing, the spacecraft's orientation is continuously adjusted to ensure that its *z-axis* points toward the Earth's center. The approach follows several steps:

1) *LVLH to ECI*
The orientation of the *LVLH* frame relative to the *ECI* frame is computed using the orbital parameters

$$\mathbf{q}_{LN} = \mathbf{q}(\Omega) \otimes (\mathbf{q}(i) \otimes \mathbf{q}(\omega + \theta)) \tag{13}$$

Since the spacecraft moves along its orbit, the true anomaly θ changes over time, which in turn affects the orientation of the LVLH relative to the ECI. At each time step, as the true anomaly changes, the quaternion $\mathbf{q}_{LN}$is updated accordingly.

2) *Body Frame to ECI*
The spacecraft's orientation with respect to the ECI frame is computed using the kinematic equations as explained in subsection 2.7, yielding the quaternion $\mathbf{q}_{BN}$.

3) *Nadir Pointing Error*
The desired orientation for Nadir pointing is achieved when the spacecraft's z-axis aligns with the Earth's center. This condition is represented by the quaternion $\mathbf{q}_{desired} = \mathbf{q}_{LN}$. The quaternion error is then computed as the "difference" between the desired and the current orientations as follows:

$$\mathbf{q}_{error} = \mathbf{q}_{desired}^{-1} \otimes \mathbf{q}_{BN} \tag{14}$$

The correct pointing is achieved when $\mathbf{q}_{error} = [0,0,0,1]^T$. Given the error in terms of quaternion, it is then possible to retrieve the roll (ϕ), pitch (Θ) and yaw (Ψ) angles of the slew (or pointing) manoeuvre to be performed [1]. Thus, it is possible to have a physical interpretation of what a quaternion error means.

$$\begin{cases} \phi = \arctan\left(\dfrac{2(q_4 q_1 + q_2 q_3)}{1 - 2(q_1^2 + q_2^2)}\right) \\ \Theta = \arcsin(2q_4 q_2 - 2q_3 q_1) \\ \Psi = \arctan\left(\dfrac{2(q_4 q_3 + q_1 q_2)}{1 - 2(q_2^2 + q_3^2)}\right) \end{cases} \tag{15}$$

Similarly, it is possible to obtain the pointing error in terms of the total rotation angle (α) required for the quaternion to achieve the correct pointing orientation as follows:

$$\alpha = 2\arctan\left(\frac{\sqrt{q_1^2 + q_2^2 + q_3^2}}{q_4}\right) \tag{16}$$

[1] Note that the following equations are valid only in last-scalar quaternion representation.

Moreover, in order for the spacecraft's z-axis to remain aligned with the Earth's center as it moves along its orbit, the spacecraft must rotate at a rate that matches its orbital angular velocity. This ensures that the spacecraft's orientation is synchronized with its orbital motion. If the spacecraft's angular velocity deviates from the orbital angular velocity, its orientation will drift, causing it to lose Nadir pointing. To quantify the difference between the desired angular velocity ($\boldsymbol{\omega}_{desired}$) and the actual angular velocity ($\boldsymbol{\omega}$), the error is defined as:

$$\boldsymbol{\omega}_{error} = \boldsymbol{\omega} - \boldsymbol{\omega}_{desired} \tag{17}$$

When correct pointing is achieved, it yields $\boldsymbol{\omega}_{error} = [0,0,0]^T$

3 Problem Abstraction

As stated in the introduction, the aim of this report is to perform an optimal control of a spacecraft moving along a Medium Earth Orbit while ensuring Nadir pointing. This optimization problem falls into the category of *Finite Horizon Optimal Control Problems (FHOCP)* which consists in the design of an input sequence that minimizes a chosen cost criterion. This is typically expressed in the time domain over a finite horizon, ranging from t (initial time) to $t+N$ (final time), where N is called prediction horizon. Moreover, state and input constraints shall be satisfied by the designed input sequence.
In this chapter, two different formalizations of the FHOCP will be presented, underlining the motivations behind such design choices. In particular:

1. A FHOCP with three components of the torque, provided by the reaction wheels, as decision variables.
2. A FHOCP with three tuning paramenters of a PID controller, appropriately designed, as decision variables.

3.1 FHOCP with torque input

The optimization problem is formulated as follows

$$\min_{U} \quad \sum_{i=0}^{N} ||\alpha(i|t)||_2^2 \tag{18a}$$

$$\text{subject to} \quad \boldsymbol{\omega}(i+1|t) = f(\boldsymbol{\omega}(i|t), \mathbf{M}(i|t)) \qquad i=0,1,...,N-1 \tag{18b}$$

$$\mathbf{q}(i+1|t) = g(\mathbf{q}(i|t), \boldsymbol{\omega}(i|t), \mathbf{M}(i|t)) \qquad i=0,1,...,N-1 \tag{18c}$$

$$\mathbf{l}_b \leq \mathbf{U} \leq \mathbf{u}_b \tag{18d}$$

where N is the simulation horizon chosen considering the orbital period (T), (18b) and (18c) represent the evolution of the dynamical and kinematic model (respectively Equation 4 and Equation 10), and U is the input sequence of the torque that should be optimized by the solver, described as follows: [2]

$$\mathbf{U} = \left[\mathbf{M}(0|t),...,\mathbf{M}(N|t)\right]^T = \left[\left[M_x(0|t), M_y(0|t), M_z(0|t)\right],...,\left[M_x(N|t), M_y(N|t), M_z(N|t)\right]\right]^T \tag{19}$$

(18d) represents the saturation constraint on the torque values. In particular it can be rewritten as

$$-M_{max} \leq M_k(i|t) \leq M_{max}, \quad i=0,...,N-1, \quad k=x,y,z \tag{20}$$

Since in Nadir pointing the main goal is to align a spacecraft to a desired fixed orientation and to maintain it throughout its orbit, the chosen cost criterion to be minimized is the sum of the squared pointing errors (18a), expressed in terms of the total rotation angle required for the quaternion to achieve the correct pointing orientation (Equation 16).

[2] In the report, letter M indicates torque, unlike in the code where letter T is used.

The solution to the optimization problem (18) produces a control action that enables the satellite to rapidly converge to the desired orientation.
Taking into account a simulation horizon N = 200 s, sampling time $t_s = 20$ s, rotation wheels $n_w = 3$ and the number of decision variables |U| ([3]) computed as:

$$|U| = n_w \times \frac{N}{t_s} \tag{21}$$

the optimization problem can be solved very efficiently: the pointing error goes to zero in a single time step (Figure 6) and the same outcome is observed for the quaternion position error (Figure 7).

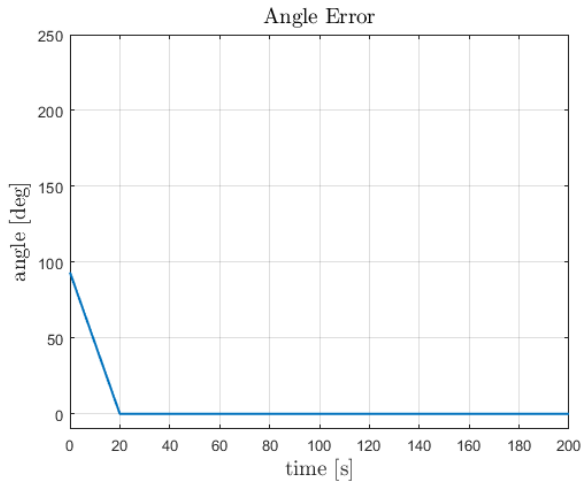

Figure 6: Pointing Error

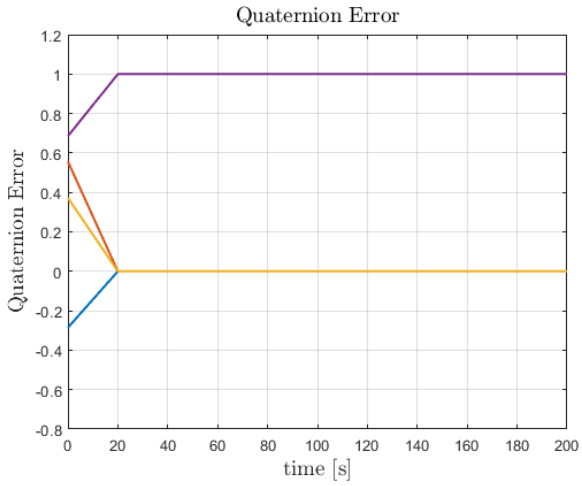

Figure 7: Quaternion Error

However, this excellent performance comes with some significant drawbacks that make it unfeasible. Since the FHOCP does not generate a control law beyond the simulation horizon, the entire duration of the orbit must be considered. As outlined in Table 3, the main issue is the large number of decision variables |U| required to control the full orbit. Additionally, applying an MPC approach to solve the FHOCP online even with a shorter prediction horizon is impractical due to the excessive computational time $t_{opt} >> t_s$ needed by the solver to find a solution. Both of these problems could be addressed by increasing the sampling time t_s, as this would reduce the number of decision variables |U| according to (21), and to easen the upper bound condition $t_{opt} < t_s$ in order to allow a feasible MPC approach. However, increasing the sampling time beyond a certain value compromises the controller's performance. Due to the large number of decision variables and t_{opt}, it becomes impossible to find a controller that can simultaneously address these issues and stabilize the satellite.

	Example	Full Orbit
N [s]	200	268704
t_{opt} [s]	845.15	
t_s [s]	20	20
|U|	30	40305

Table 3: Main drawbacks of FHOCP with input torque and $t_s = 20$ s

Given these challenges, a parameterized input approach is necessary to reduce computational time and make the problem solvable.

[3] In the report $|A|$ refers to the cardinality of A

3.2 FHOCP with PID controller

In order to overcome the problems mentioned above, a PID controller is introduced to perform the spacecraft attitude control. In particular, simulations of a PID controller action are run for a large enough finite horizon in order to capture model dynamics and then to optimally tune the PID parameters.

The optimization problem is formulated as follows

$$\min_{PID} \sum_{i=0}^{N} \lambda_1 ||\mathbf{q_{error}}(i|t)||_2^2 + \lambda_2 ||\boldsymbol{\omega_{error}}(i|t)||_2^2 + p||\mathbf{M_{act}}(i|t)||_2^2 \tag{22a}$$

$$\text{subject to} \quad \boldsymbol{\omega}(i+1|t) = f(\boldsymbol{\omega}(i|t), \mathbf{M}(i|t), \mathbf{PID}(i|t)) \qquad i=0,1,...,N-1 \tag{22b}$$

$$\mathbf{q}(i+1|t) = g(\mathbf{q}(i|t), \boldsymbol{\omega}(i|t), \mathbf{M}(i|t), \mathbf{PID}(i|t)) \qquad i=0,1,...,N-1 \tag{22c}$$

$$\mathbf{M_{act}}(i+1|t) = h(\mathbf{M_{act}}(i|t), \boldsymbol{\omega}(i|t), \mathbf{q}(i|t), \mathbf{PID}(i|t)) \qquad i=0,1,...,N-1 \tag{22d}$$

$$\mathbf{l}_{b_M} \leq \mathbf{M_{act}} \leq \mathbf{u}_{b_M} \tag{22e}$$

$$\mathbf{l}_{b_{pid}} \leq \mathbf{PID} \leq \mathbf{u}_{b_{pid}} \tag{22f}$$

where (22b), (22c) and (22d) represent the evolution of the dynamical and kinematic model of the system and PID is the vector composed by the tuning parameter of the PID controller, defined as

$$\mathbf{PID} = \begin{bmatrix} k_p & k_i & k_d \end{bmatrix}^T \tag{23}$$

(22e) is the saturation constraint of the control torque already discussed in subsection 2.6 where

$$\mathbf{l}_{b_M} = -10^{-2} \tag{24}$$

$$\mathbf{u}_{b_M} = 10^{-2} \tag{25}$$

Then a series of parameters are introduced, regarding weights and penalties in cost function and lower and upper constraints. These quantities are found by trial and error approach running multiple simulation and manually tuning them in order to find realistic values, for this specific problem, that could lead to convergence.

(22f) represent lower and upper bounds on the values of the PID tuning parameters with

$$\mathbf{l}_{b_{pid}} = \begin{bmatrix} 10^{-3}, & 10^{-5}, & 10^{-7} \end{bmatrix}^T \tag{26}$$

$$\mathbf{u}_{b_{pid}} = \begin{bmatrix} 10, & 0.01, & 1 \end{bmatrix}^T \tag{27}$$

Particular attention should be given to the upper bound of k_d. It is important to note that if the derivative parameter exceeds a certain threshold during the optimization process, the simulation will stop due to integration and numerical issues.

(22a) is the cost function, which is based on the sum of the squared errors of the quaternion $\mathbf{q_{error}}$, the angular velocity $\boldsymbol{\omega_{error}}$, and a penalty on the input torque $\mathbf{M_{act}}$, which is included to reduce the control effort. This is done in addition to the torque constraints, which simply restrict the torque to remain within a specified range. The weights and penalty are as follows:

$$\lambda = \begin{bmatrix} \lambda_1 \\ \lambda_2 \end{bmatrix} = \begin{bmatrix} 10^8 \\ 10^4 \end{bmatrix} \tag{28}$$

$$p = 10^2 \tag{29}$$

The process to find these parameters is composed by two steps. The first aims to normalize the contribution of each term of the cost function, ensuring that all the terms are of the same order of magnitude and preventing any single term from disproportionately affecting the cost function. The second step actually weights the relative contribution of each term.

3.2.1 PID design and implementation

The PID controller consists of a proportional unit P, an integral unit I and a differential unit D. The classical PID control law is expressed in time domain as

$$u(t) = k_p e(t) + k_i \int_0^t e(t)dt + k_d \frac{de(t)}{dt} \tag{30}$$

that can be rewritten in Laplace domain as

$$U(s) = (k_p + k_i \frac{1}{s} + k_d s)E(s) \tag{31}$$

where s is the Laplace variable, k_p,k_i,k_d are respectively the proportional, integral and derivative gains and $E(s)$ is the output error of the system and is also the input of the PID controller.
In general, the proportional gain k_p weights the current value of the error, the integral gain k_i relates to the cumulative sum of past errors and the derivative gain k_d tends to anticipate the future value of the error.
The PID control law used to perform the spacecraft attitude control is expressed as [9]

$$\mathbf{M_{act}} = I\dot{\boldsymbol{\omega}} - I\boldsymbol{\omega} \times \boldsymbol{\omega} - \mathbf{M}_{dist} = -k_p \mathbf{q_{error}} - k_d \boldsymbol{\omega}_{error} \tag{32}$$

The integral action has been included too, so the new control law becomes as follows:

$$\mathbf{M_{act}} = -k_p \mathbf{q_{error}} - k_d \boldsymbol{\omega}_{error} - k_i \frac{\mathbf{q_{error}}}{s} \tag{33}$$

where $\mathbf{q_{error}}$ and $\boldsymbol{\omega}_{error}$ are the output errors of the model and the inputs of the PID controller. Since the time derivative of the quaternion error does not have a straightforward physical interpretation, the derivative action of the PID controller is instead applied to the angular velocity error. This choice is justified because angular velocity is directly related to the rate of change of attitude and can be linked to quaternion kinematics, as shown in Equation 10. By using $\boldsymbol{\omega}_{error}$ for the derivative term, the controller effectively accounts for rapid attitude variations while maintaining a physically meaningful and well-behaved control response.

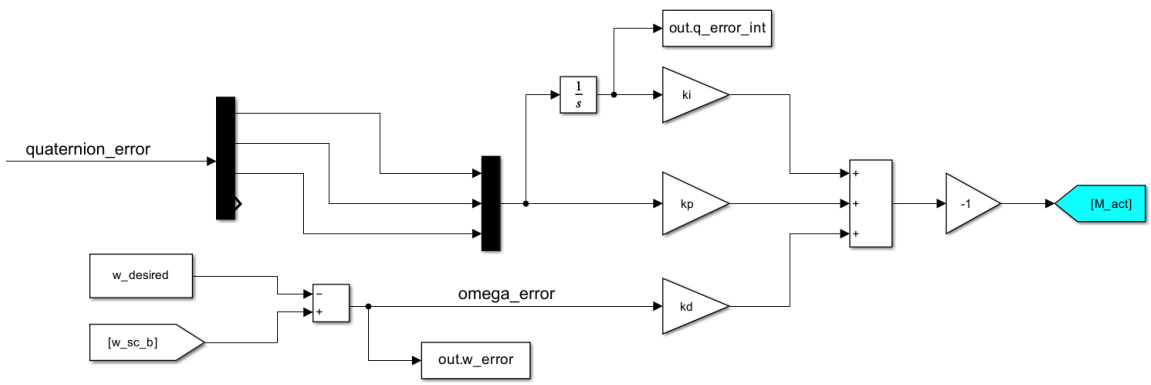

Figure 8: PID implemented in simulink

3.3 Final considerations

The FHOCP with torque input, as described in subsection 3.1, is capable of finding the optimal solution for achieving the fastest response in almost a single time step. However, due to the long duration of the simulation horizon, the related number of decision variables and the significant computation time required by the solver, implementing this type of controller for this problem is not feasible. A possible solution presented in subsection 3.2 is to parametrize the input following a PID control law. This parametrization reduces the decision variables to just 3, regardless of the simulation horizon length or the sampling time. Therefore, the computational effort is markedly reduced, as it is shown in Table 5. Moreover, the PID is an output feedback controller, benefiting from the advantages of a closed-loop configuration, while the FHOCP with input torque is in open loop approach. Nevertheless, due to the input parametrization, the degrees of freedom are reduced, and it is, in some way, suboptimal.

4 Optimization Procedure

4.1 Solver Selection for Optimization

Choosing the appropriate solver requires a thorough understanding of the system's characteristics. In this case, the system dynamics is nonlinear, meaning that the optimization solver must be capable of handling nonlinear constraints. Additionally, there are limits on both the PID coefficients and on the torque, due to actuator power limitations.
Given these conditions, `fmincon` is a natural choice for solving this problem, as it is designed to handle constrained nonlinear optimization. However, an important challenge is that the problem is non-convex, meaning that solvers may converge to a local minimum rather than a globally optimal solution. To mitigate this, a specific optimization strategy was implemented to ensure convergence to a satisfactory solution.

4.2 Algorithms overview

Solver `fmincon` offers several algorithms, each with different characteristics and suitability for different types of problems. The options include:

- Interior-Point (default)
- Trust-Region-Reflective
- SQP (Sequential Quadratic Programming)
- SQP-Legacy
- Active-Set

Interior-Point Method The interior point algorithm is well suited for large-scale problems due to its low memory usage and fast computation. However, it relies on a barrier function, which prevents iterations from reaching the constraints, potentially leading to less accurate solutions near the feasible region. It is particularly useful for problems with many inequality constraints, as it can solve quadratic programs with hundreds of variables efficiently [2].
Despite these advantages, this method is not ideal for the current problem due to its potential loss of accuracy at constraint boundaries.

Trust-Region-Reflective This method requires explicit gradient information and can only handle bound constraints or linear equality constraints, but not both [3]. Since the present problem involves both bounds and non-linear constraints, this method is not suitable.

Active-Set Method The active-set algorithm is efficient when the number of inequality constraints is small, as it can take large steps toward a solution. However, it tends to scale poorly with the size of the problem and can be computationally expensive when dealing with a large number of constraints [6]. While it performs well in some nonlinear problems, it is not ideal for this case due to a large number of inequality constraints for torque.

SQP (Sequential Quadratic Programming) The `fmincon` 'sqp' and 'sqp-legacy' algorithms are similar to the 'active-set' algorithm. The most important differences between the sqp and the active-set algorithms are [7]:

- Strict Feasibility With Respect to Bounds.
- Robustness to Non-Double Results.
- Refactored Linear Algebra Routines.
- Reformulated Feasibility Routines

4.3 Evaluation of Optimization Algorithms for Given Nonlinear Constrained Problem

This section presents a comparative analysis of optimization algorithms applied to a nonlinear constrained control problem. The optimization aims to determine optimal PID parameters, considering system constraints such as actuator torque limits and stability criteria. The analysis is conducted 15 times under consistent initial conditions, ensuring that variations in results arise solely from algorithmic differences rather than differences in initial states. Moreover, other settings of solver were the same for each algorithm Table 4.

Option	**Value**
MaxIterations	500
TolCon	$1{\cdot}10^{-10}$
MaxFunctionEvaluations	5000
OptimalityTolerance	$1{\cdot}10^{-8}$
StepTolerance	$1{\cdot}10^{-8}$
ConstraintTolerance	$1{\cdot}10^{-8}$
FiniteDifferenceType	central
HessianApproximation	bfgs (default)

Table 4: Common algorithms settings

The evaluation focuses on the following performance metrics:

- Convergence efficiency (number of iterations required to reach an optimal solution).
- Computational complexity (number of function evaluations).
- Computational time (total solver runtime).
- Solution quality (final objective function value)

The algorithms examined in this study include:

- SQP (Sequential Quadratic Programming).
- Interior-Point Method.
- Active-Set Method.

Each of these methods operates differently in handling constraints, updating optimization variables, and determining step directions. The results presented [4] in this section provide a data-driven basis for selecting the most appropriate algorithm for this class of problems (Figure 9).

[4]The file "optimization_results_full_algo.txt" stores all the results used for analysis.

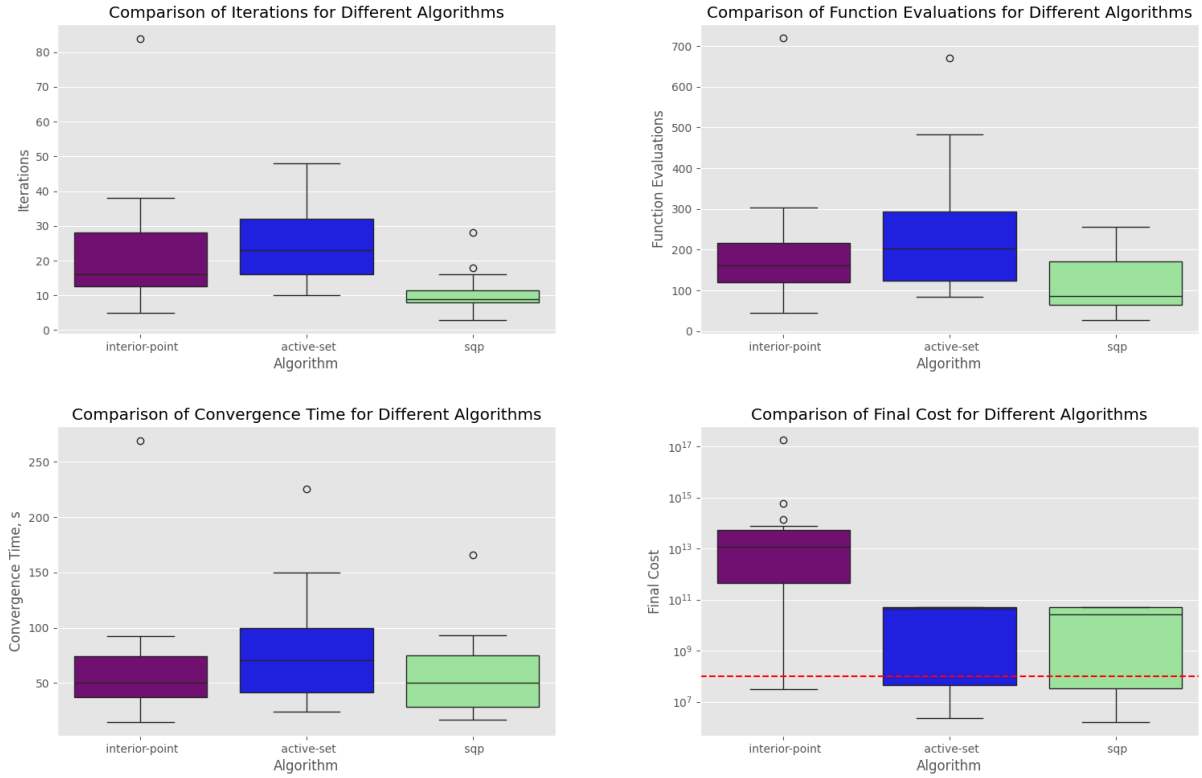

Figure 9: Comparison of Different Algorithms

4.3.1 Iterations Required for Convergence

The number of iterations required for an algorithm to reach an optimal solution is a direct indicator of its efficiency. A lower iteration count suggests a solver's ability to adjust the optimization variables effectively with fewer computational steps. (Figure 9) shows that:

- SQP requires the fewest iterations, demonstrating a more direct path to convergence.
- Interior-point requires a moderate number of iterations, suggesting that its constraint-handling mechanism slows down convergence compared to SQP.
- Active-set requires the highest number of iterations, implying that its search process is less efficient in this problem formulation.

The data suggest that SQP is the most efficient in terms of iteration count, while active-set is the least efficient.

4.3.2 Function Evaluations

Function evaluations measure how frequently the solver must evaluate the cost function and constraints before reaching a solution. A high number of function evaluations may indicate excessive computational effort.

- SQP has the lowest number of function evaluations, indicating that it optimizes efficiently.
- Interior-point requires more function evaluations than SQP but significantly fewer than active-set.
- Active-set has the highest function evaluation count, confirming its inefficiency in searching the solution space.

4.3.3 Computational Time

In principle results are consistent with two previous metrics:

- SQP and interior-point demonstrate the shortest computation times, making them suitable for time-sensitive applications.
- Active-set requires significantly longer computation times, which limits its practical use in real-time applications.
- The presence of outliers in interior-point and active-set indicates that these methods occasionally experience unexpectedly long convergence times.

These results suggest that SQP is the most reliable option when computational efficiency is a primary concern.

4.3.4 Final Cost (Objective Function Value)

The final cost function value indicates how well each algorithm minimizes the objective function.

- SQP achieves the lowest final cost, indicating that it finds slightly better solutions in terms of objective function minimization.
- Active-set produces solutions with slightly higher final cost values, though still within an acceptable range.
- Interior-point has the highest variance in final cost values, suggesting that it does not consistently find the best solution.

Since the problem is non-convex, multiple solutions result in unacceptable costs. Validation was performed using simulation with the obtained PID coefficients, identifying the minimum acceptable cost function value as 1e8. This threshold is represented by the red line on the graphs (Figure 9). Notably, for the interior-point algorithm, this value falls outside the interquartile range. While other algorithms perform slightly better, the acceptable solution remains at the edge of the interquartile range.

4.3.5 Conclusion

Given these results, SQP is the preferred algorithm for this problem, as it provides the best balance between computational efficiency and solution quality. While active-set achieves the same cost function values, it does so at the expense of excessive computational effort and long solving times. Interior-point exhibits moderate performance but lacks consistency in final solution quality.

4.4 Finite Difference Type Comparison: Forward vs. Central Differentiation

4.4.1 Introduction

The solver `fmincon` provides two options for finite difference approximation:

- **Forward Difference:** Uses a single perturbed point to estimate gradients.
- **Central Difference:** Uses two perturbed points (one forward and one backward) to obtain a more accurate derivative approximation.

This section analyzes the effect of `FiniteDifferenceType` (`forward` vs. `central`) on the optimization process, using **SQP**, which was previously identified as the most suitable algorithm. The comparison is conducted 10 times under the same initial conditions to ensure that variations in performance arise solely due to differences in the finite difference method. Other settings of the solver the same as in the previous chapter Table 4, metrics are also the same. Finite Difference Step Sizes are default, thus optimal depending on the fintite difference type. (Figure 10) represents the results of analysis[5].

[5]The file "optimization_results_diff.txt" stores all the results used for analysis.

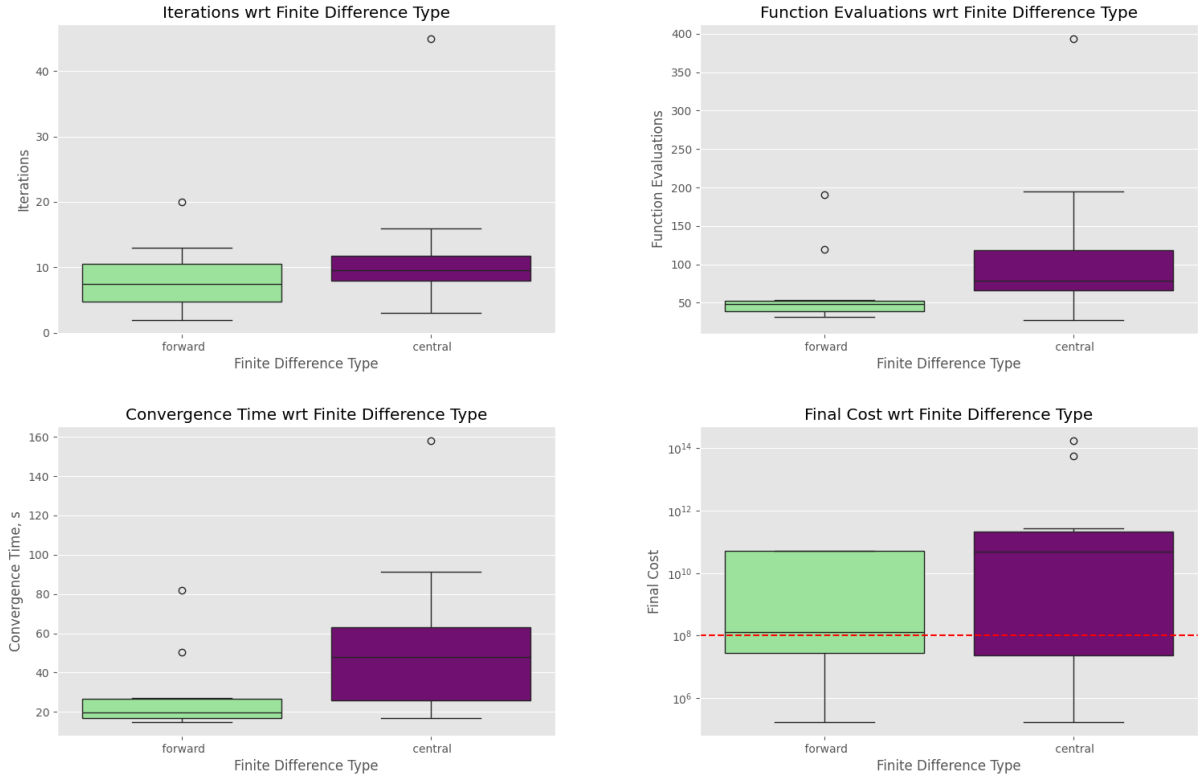

Figure 10: Comparison of Finite Difference Type

4.4.2 Iterations Required for Convergence

The number of iterations is almost similar between forward and central differences, with central difference slightly higher on average. Central difference has lower variance, indicating more predictable performance. At the same time Central difference exhibits long whiskers, suggesting higher dataspread. These results suggest that forward difference is marginally more efficient in terms of iteration count, making it a preferable choice when minimizing solver runtime.

4.4.3 Function Evaluations

Forward difference requires significantly fewer function evaluations compared to central difference. Central difference exhibits high variance, with some cases requiring up to 200 function evaluations, while forward difference remains below 50 in most cases. The increased function evaluations for central difference result from its use of two points per gradient estimation, effectively doubling the number of function calls. This confirms that forward difference is computationally more efficient, making it suitable for problems where function evaluations are expensive.

4.4.4 Computational Time

The increased computation time associated with the central difference method arises from its higher number of function evaluations. Since each function call adds to the overall computational cost, the central difference method requires approximately twice as many evaluations as the forward difference method. This outcome aligns with the previously discussed metrics and the inherent nature of the method.

4.4.5 Final Cost (Objective Function Value)

In this case, the results are less straightforward. Both methods yield similar average final cost values, indicating that the forward difference method does not significantly degrade solution quality. Notably, the median cost for the forward difference method is much lower and closer to the acceptable threshold, whereas the median for the central difference method is considerably higher. Additionally, the interquartile range for the central difference method is larger, and it exhibits some outliers. But what might explain this behavior? One possible reason is that in non-convex optimization, less precise gradient estimates can sometimes help escape poor local minima, leading to improved final cost values.

4.4.6 Conclusion

Overall, these findings suggest that the forward difference method may be preferable, as the central difference method tends to produce worse solutions on average, while its increased computational cost does not provide a clear advantage.

4.5 Other solver settings

The influence of solver settings related to tolerances and constraints on iterations and function evaluations was not examined in this study. These parameters were primarily selected to be slightly more restrictive than the default values to improve the accuracy of the solution without significantly increasing computational time. In particular, the chosen settings met both criteria effectively.

4.6 Handling Non-Convexity in Optimization

As previously mentioned, the optimization problem under consideration is non-convex, whereas the solver `fmincon` is designed for convex optimization. During the validation process, it was observed that achieving a final cost below 1e8 (as referenced in Section 4.3.4) is sufficient. To address this issue, fmincon is executed iteratively in a loop, modifying the initial guess for the PID coefficients randomly in each iteration until a satisfactory cost function value is attained. An example of the implemented procedure is provided below:

```
while fxstar >= 1e8
    PID0 = lb + (ub - lb) .* rand(1,3);
    [xstar, fxstar, k, exitflag, xsequence] = fmincon(...);
end
```

5 Results

The simulation is performed in Simulink [6] over the entire orbital period with a time step of 10 seconds, corresponding to a 0.1 Hz sampling rate. This time step is chosen to ensure the system operates within the typical control bandwidth for space missions, which ranges from 0.01 Hz to 0.1 Hz (Ref. [5]). Smaller time steps were found to result in instability, likely due to overreaction to small errors or increased sensitivity to numerical inaccuracies. With the 10-second time step, the system demonstrates improved stability, avoiding excessive oscillations and ensuring effective tracking performance.
Another important aspect is the choice of initial conditions for the simulation. If the initial conditions are chosen improperly or unrealistically, the simulation may not accurately represent the spacecraft's performance, leading to suboptimal or even incorrect results. The initial quaternion, representing the spacecraft's initial attitude, is randomly generated within the bounds of typical slew maneuvers, involving rotations up to 90° in roll, pitch, and yaw as mentioned in section 2. To ensure repeatability and coherence of the results presented in this report, the specific initial quaternion used is

$$\mathbf{q}_0 = \begin{bmatrix} -0.1973 & -0.4659 & 0.6160 & 0.6037 \end{bmatrix}^T$$

By computing the error, it can be observed that, in order to achieve the desired pointing, the spacecraft needs to perform the following rotations:

$$\phi = 5.9495 \quad deg \qquad \Theta = 77.3356 \quad deg \qquad \Psi = 61.743 \quad deg$$

Regarding the initial angular velocity, since the detumbling phase is not considered in this simulation, it is set to a small value. Typically, the initial angular velocity is chosen to be close to zero to reflect a spacecraft that is already stabilized or nearly at rest in its orientation. In this simulation, the initial angular velocity is set equal to the desired angular velocity.

[6] Fast Restart allows Simulink to retain the compiled state of the model, significantly accelerating the simulation, especially when frequently evaluating gradients and constraints, which would otherwise require repeated model recompilation and so higher computational time. (Ref [10]).

The optimal values obtained from the `fmincon` optimization routine are:

$$k_p = 1.1449 \cdot 10^{-2} \qquad k_d = 1.0000 \cdot 10^{-2} \qquad k_i = 1.2000 \cdot 10^{-7} \tag{34}$$

Additional results are provided in Table 5.

Parameter	**Value**
Algorithm	SQP
Finite Difference Type	Forward
Final Cost (Fval)	$6.4996 \cdot 10^7$
Number of Iterations	9
Final Function Evaluations	74
Final Relative Constraint Violation	0.000
Computational time [s]	30.810

Table 5: Final Optimization Results from fmincon

It can be observed that the computational time for the parametrized FHOCP is significantly lower than that of Torque input optimization (see Table 3). This further emphasizes the advantage of formulating the problem using parametrized optimization.

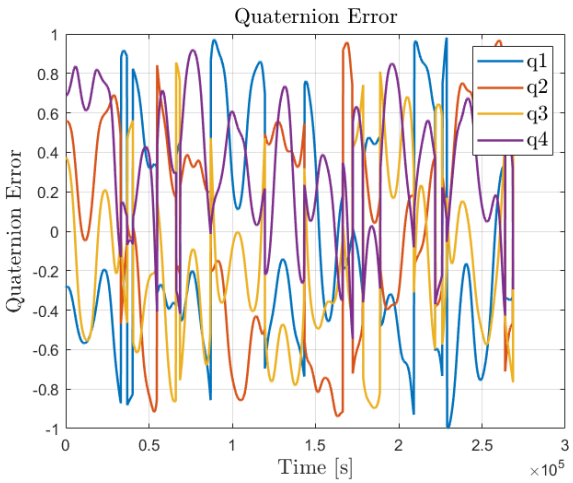

Figure 11: Quaternion error without control

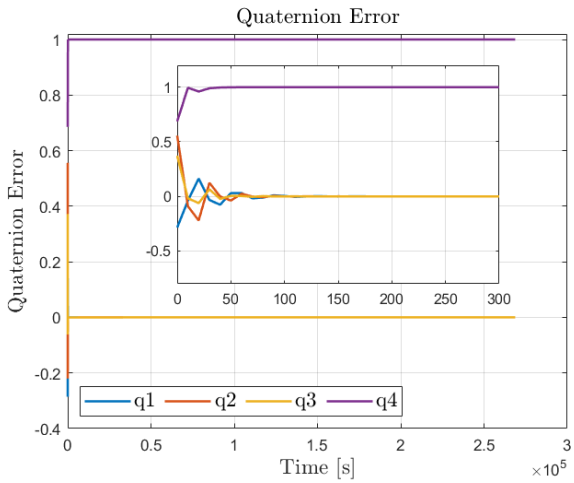

Figure 12: Quaternion error with PID

Figure 11 and Figure 12 show the evolution of quaternion errors before and after the control torque is applied. As observed, the system achieves rapid convergence, within less than 100 seconds, ensuring that the spacecraft completes the desired slew maneuver and accurately reaches the Nadir pointing. Once the correct pointing is established, the errors remain at zero, confirming that the spacecraft maintains precise nadir pointing throughout the entire orbit, effectively stabilizing its orientation.

In line with the evolution of the quaternion errors, the pointing angle, in terms of the total rotation performed by the spacecraft, converges to zero, as shown in Figure 14. It is evident that the spacecraft has completed nearly 100 degrees of rotation, successfully executing a significant slew maneuver. In Figure 13, it can be observed a non-controlled scenario, where the pointing error oscillates as the spacecraft moves, continuously changing its orientation.

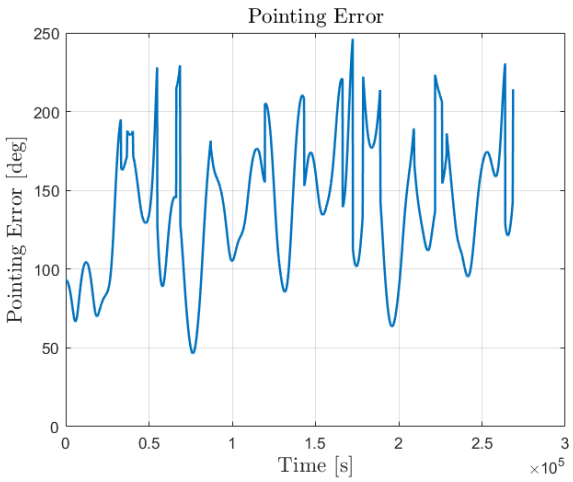

Figure 13: Pointing error without control

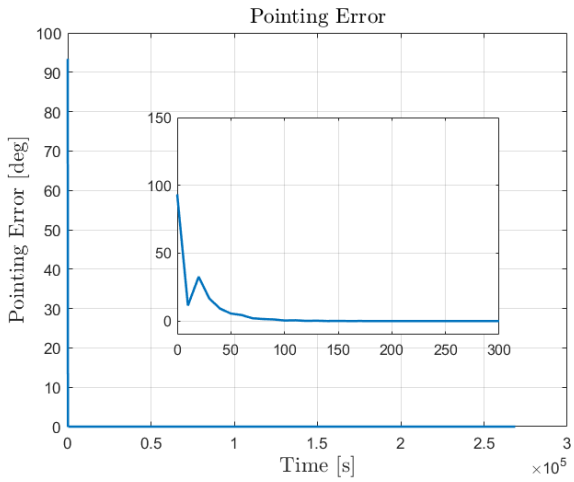

Figure 14: Pointing Error with PID

The spacecraft angular velocity follows a similar trend (Figure 16), rapidly converging to the desired value. Initially, slight oscillations occur, with the velocity peaking at 0.1 rad/s before stabilizing, which is a realistic value for a slew maneuver. The penalty in the cost function effectively minimizes these oscillations, ensuring minimal fluctuations in velocity.

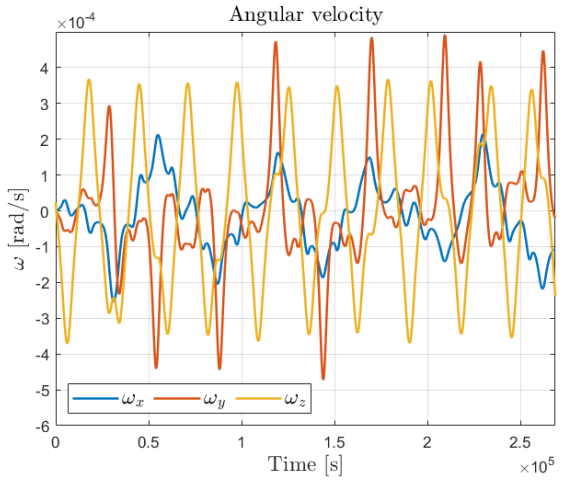

Figure 15: Angular velocity without control

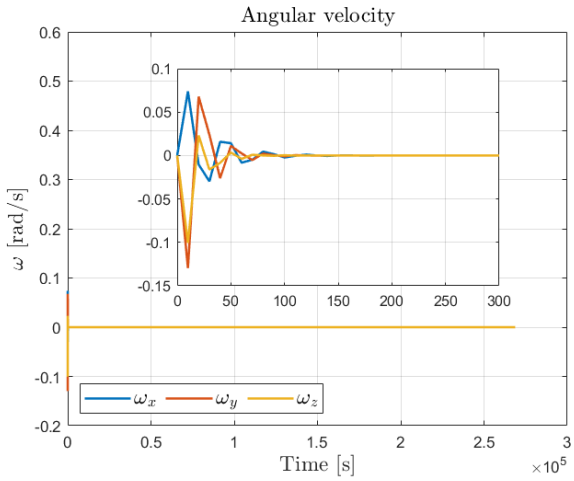

Figure 16: Angular velocity with PID

The control torque is illustrated in Figure 17. As expected, nonlinear constraints related to saturation are fully respected, ensuring efficient control. The torque decreases to near zero once the desired pointing is achieved, indicating that after the slew maneuver, the spacecraft requires minimal control torque to maintain the Nadir pointing. The PID controller efficiently stabilizes the spacecraft, keeping precise Nadir orientation with minimal effort, demonstrating the effectiveness of the control strategy throughout the entire orbit.

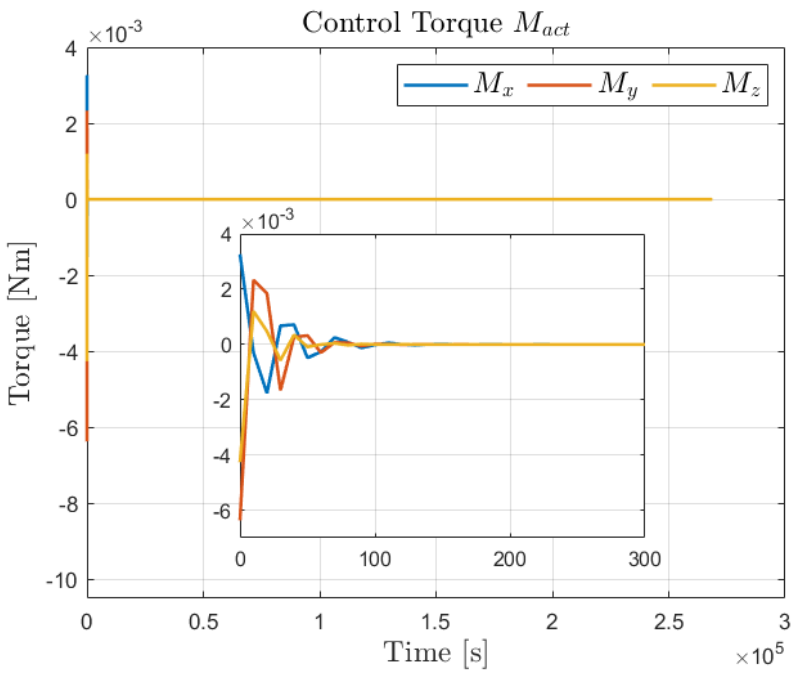

Figure 17: Control Torque

References

[1] AAC Clyde Space. *RW400 CubeSat Reaction Wheels*. URL: https://www.aac-clyde.space/what-we-do/space-products-components/adcs/rw400.

[2] Richard H. Byrd, Mary E. Hribar, and Jorge Nocedal. "An Interior Point Algorithm for Large-Scale Nonlinear Programming". In: *SIAM Journal on Optimization* 9.4 (1999), pp. 877–900. DOI: 10.1137/S1052623497325107.

[3] A. R. Conn, N. I. M. Gould, and Philippe L. Toint. *Trust-Region Methods*. MPS/SIAM Series on Optimization. SIAM and MPS, 2000. DOI: 10.1137/1.9780898719857.

[4] Howard D. Curtis. *Orbital Mechanics for Engineering Students*. 4th. Elsevier, 2021.

[5] Cornelius Dennehy and Oscar S. Alvarez-Salazar. *Spacecraft Micro-Vibration: A Survey of Problems, Experiences, Potential Solutions, and Some Lessons Learned*. Tech. rep. NASA/TM-2018-220075. Hampton, Virginia and Pasadena, California: NASA Langley Research Center and Jet Propulsion Laboratory, July 2018. URL: https://ntrs.nasa.gov/api/citations/20180006315/downloads/20180006315.pdf.

[6] Philip E. Gill, Walter Murray, and Margaret H. Wright. *Practical Optimization*. London: Academic Press, 1981.

[7] MathWorks. *Constrained nonlinear optimization algorithms*. Accessed: 2025-02-01. n.d. URL: https://www.mathworks.com/help/optim/ug/constrained-nonlinear-optimization-algorithms.html#bsgppl4.

[8] Santosh Ratan and Kuk Byun. "Slewing of a Spinning Satellite by Reaction Wheels". In: *SpaceOps 2008 Conference*. 2008, p. 3225.

[9] Yihe Shan, Lurui Xia, and Sen Li. "Design and Simulation of Satellite Attitude Control Algorithm Based on PID". In: *Journal of Physics: Conference Series*. Vol. 2355. 1. IOP Publishing. 2022, p. 012035.

[10] The MathWorks, Inc. *Get Started with Fast Restart*. MathWorks. Natick, MA, 2025. URL: https://www.mathworks.com/help/simulink/ug/fast-restart-workflow.html.